Object-Oriented Programming in Java

Java Exercises

Exercises

28 hands-on coding exercises on the five slide decks J01–J04: the Java environment, Java basics, characters and strings, JUnit 5, inheritance and interfaces. From 2.7 on, every exercise is a small exam in the style of the course's past exams: a facade class, requirements R1, R2, ..., exact formats and error messages. Difficulty rises inside each part, from ★ warm-up to ★★★ challenge.

Contents

How to use this sheet **2**

Part 1 – The Java environment (J01) **3**
1.1 ★ Hello, command line . . . 3
1.2 ★ Fix the conventions . . . 3
1.3 ★★ Jar file and magic number . . . 3

Part 2 – Java basics (J02) **4**
2.1 ★ Calendar utilities . . . 4
2.2 ★ Counter and Car . . . 4
2.3 ★★ Encapsulated bank account . . . 5
2.4 ★★ A shop in two packages . . . 6
2.5 ★★ Number tools: wrappers, Scanner, casts . . . 6
2.6 ★★ Array tools . . . 7
2.7 ★★ A box office: static, final, factory methods, singleton . . . 8
2.8 ★★ Food delivery: enum, record and a checked exception . . . 9
2.9 ★★★ A trail race: nested classes . . . 11

Part 3 – Characters and strings (J02a) **13**
3.1 ★ Job applications: text utilities . . . 13
3.2 ★★ A restaurant: formatted bills . . . 14
3.3 ★★ Supplier bids: StringBuilder, speed and the string pool . . . 15
3.4 ★★★ Importing contacts: encodings . . . 16

Part 4 – JUnit 5 (J03) **18**
4.1 ★ Exam-style tests for a parking lot . . . 18
4.2 ★★ Testing the food delivery of 2.8, then a suite . . . 19
4.3 ★★★ Bug reproduction and TDD . . . 20

Part 5 – Inheritance, interfaces, lambdas (J04) **22**
5.1 ★ Bank accounts: inheritance . . . 22
5.2 ★★ Customers and orders: equals, hashCode, toString, clone . . . 23
5.3 ★★ Bank operations: abstract classes and Template Method . . . 24
5.4 ★★ Pricing rules: interfaces, static and default methods . . . 25
5.5 ★★★ Menu combos: Composite . . . 26
5.6 ★★★ Restaurant chain reports: Comparable, Comparator, Strategy . . . 27
5.7 ★★★ A train timetable: Iterable and Iterator . . . 29
5.8 ★★★ Order statistics: functional interfaces, lambdas, method references . . . 30
5.9 ★★★ Delivery notifications: Observer by composition (code + test) . . . 31

How to use this sheet

28 coding exercises cover the topics of the five slide decks, in deck order, and get harder inside each part: ★ warm-up, ★★ core, ★★★ challenge. Every exercise asks you to write Java code; most end with a `main` (or a test class) and a **sample run** to check your program against. Solutions are in the companion PDF, *Java Exercises – Solutions*, and the theory each exercise needs is in *Java Exercises – Theory notes*.

From 2.7 on, most exercises are small exams, built like the past exams of the course (Food Delivery, Restaurants, Bank System, Trail, Job Applications…). They have a *facade* class (the one the exam's tests talk to) in their own package, often requirements R1, R2, … to implement in order, and exact output formats and error messages. As at the exam:

- numbering starts from 1 in every facade object;
- errors are signalled with a checked exception of the package (from 2.8);
- lists come back sorted as asked.

The collections (`List`, `Map`) are not in the slides yet, so keep the data in arrays; the capacity given in the text is enough (when an array is full, throw `IllegalStateException`). Part 4 has you write the tests, the way the exam's `TestR1_…` classes are written.

Use Java 25 (the slides use `IO.println` and `void main()`); Part 4 needs JUnit 5. One package per exercise keeps names apart, e.g. `ex2_3` for exercise 2.3.

Where an exercise gives code followed by a sample run, that code is the body of `main`. From 2.7 on, `main` goes in a class `Main` of the exercise's package. In 2.5 and 2.6 the code calls static methods without a class name, so put that `main` inside the class you write; in 2.1–2.4 follow the text. When the demo calls methods that throw a checked exception without catching it, the text says *main declared with: throws* …. Part 1 uses the default package (no `package` line).

Part	Slide deck	Exercises	Topics
1	J01 Java Environment	1.1–1.3	javac/java, command-line arguments, Java 25 `void main()` + `IO`, coding conventions, jar + manifest, 0xCAFEBABE
2	J02 Java Basics	2.1–2.9	control statements, switch on strings, boolean operators, classes, constructors, overloading, `this`, method chaining, aliasing vs copy, static members, encapsulation, final, constants, packages and visibility, wrappers, autoboxing, Scanner, casts, arrays, jagged arrays, varargs, `Arrays`, utility classes, static import, factory methods, singleton, static blocks, immutability, enum, record, static nested / inner / local classes; the exam's facade, per-object counters, checked exceptions
3	J02a Chars and Strings	3.1–3.4	`Character`, String methods, `split`, `parseInt`, `compareTo`, format/formatted, rounding, StringBuilder/StringBuffer and their speed, sorted arrays, string pool and `==`, `Charset`, UTF-8 / UTF-16 / ISO-8859-1, mojibake, code points
4	J03 JUnit	4.1–4.3	exam-style test classes, @Test, assertions with messages and tolerance, assertThrows, @BeforeEach/@AfterEach/@BeforeAll/@AfterAll, suites, @Disabled, assumptions, TDD, bug reproduction

Part	Slide deck	Exercises	Topics
5	J04 Inheritance	5.1–5.9	extends, override, super, protected hooks, polymorphism, dynamic binding, casts, instanceof, final methods, toString/equals/hashCode, Cloneable, abstract classes, Template Method, interfaces, static/default methods, anonymous classes, Composite, flag interfaces, Comparable/Comparator, Strategy, Iterable/Iterator, views, functional interfaces, lambdas, method references, Observer, composition

Part 1 – The Java environment (J01)

1.1 ★ Hello, command line

Write `Greeter.java`: for each command-line argument it prints `Hello, <arg>!` on its own line. With no arguments it asks for a name with `IO.readln("Name? ")` and greets it, or greets `world` when the user just presses Enter. Compile it with `javac` and run it from the terminal. Then write `GreeterCompact.java`, the same program in the Java 25 form from the slides (no `public static`).

Sample run:

```
$ java Greeter Ada Linus
Hello, Ada!
Hello, Linus!
$ java Greeter
Name? Gabriele
Hello, Gabriele!
$ java Greeter
Name?
Hello, world!
```

1.2 ★ Fix the conventions

Rename everything in this class so it follows the course's coding conventions (same behaviour), and save it in the right file.

```
public class bank_account {
    static final double interestrate = 0.02;
    private double Balance;
    public void Deposit_Money(double AMOUNT) { Balance += AMOUNT; }
    public double get_balance() { return Balance; }
}
```

1.3 ★★ Jar file and magic number

1. Package `Greeter` into `greeter.jar` with a manifest, so that `java -jar greeter.jar Ada` prints `Hello, Ada!`. Also list the jar's contents, and run `Greeter` from the jar through the classpath, without the manifest.
2. Write `MagicNumber.java`: it reads the file named by `args[0]`, prints its first 4 bytes in hexadecimal, and says whether it is a Java class file (class files start with `CAFEBABE`). Read the file with `byte[] data = java.nio.file.Files.readAllBytes(java.nio.file.Path.of(args[0]));` and declare `main(...) throws Exception`. Careful: Java bytes are signed.

Sample run (the `.java` line shows the codes of its first four characters, `publ`):

```
$ java MagicNumber Greeter.class
Greeter.class starts with CAFEBABE
It is a Java class file
$ java MagicNumber Greeter.java
Greeter.java starts with 7075626C
It is not a Java class file
```

Part 2 – Java basics (J02)

2.1 ★ Calendar utilities

Write a class `CalendarUtils` with these static methods:

- `String season(String month)`: a switch on strings with grouped cases – december–february → `"winter"`, march–may → `"spring"`, june–august → `"summer"`, september–november → `"autumn"`, anything else → `"unknown"`. Case must not matter.
- `boolean isLeap(int year)`: divisible by 4 and not by 100, or divisible by 400 – one boolean expression, no `if`.
- `int daysInMonth(String month, int year)`: 28/29/30/31, or -1 for an unknown month.
- `int daysInYear(int year)`: a loop over an array of the 12 month names.
- `int nextLeapYear(int year)`: the first leap year after `year`, found with a `while` loop.
- `main`: for each year on the command line (converted with `Integer.parseInt`; 2023, 2024, 1900, 2000 when there are none), print its number of days and mark leap years; then print the next leap year after 2025 and the seasons of `"JULY"` and `"Smarch"`.

Sample run, no arguments:

```
2023: 365 days
2024: 366 days (leap)
1900: 365 days
2000: 366 days (leap)
Next leap year after 2025: 2028
JULY -> summer, Smarch -> unknown
```

2.2 ★ Counter and Car

1. Complete the slides' `Counter` (they call `decrement(7)` but never define it): a private `int value`; constructors `Counter()` and `Counter(int start)`, the first delegating with `this(...)` (written inside a constructor, `this(args)` calls another constructor of the same class); `reset()`, `increment(int by)`, `decrement(int by)` and `print()` (prints the value with `System.out.println`), all returning `this`; a query `getValue()`.
2. Write `Car` with private `brand`, `color`, `turnedOn` and:
   - constructors `Car()` (a red Ferrari), `Car(String brand)` (white), `Car(String brand, String color)`, and a copy constructor `Car(Car other)` that builds an independent copy; all of them go through the two-argument one with `this(...)`;
   - a private static counter of the cars built, with a static getter `getCountBuiltCars()`;
   - `turnOn()`, `paint(String color)` (use `this` to tell the attribute from the parameter) and an overloaded `paint()` that paints it white, `isOn()`, getters, `sameAs(Car other)` comparing brand and color, and `toString()` like `Ferrari red (off)`.

Run this `main`:

```
new Counter(3).print().reset().print().increment(10).print().decrement(7).print();

Car a = new Car();
Car b = a;
Car c = new Car(a);
b.paint("yellow");
c.turnOn();
IO.println(a + " | " + b + " | " + c);
IO.println((a == b) + " " + (a == c) + " " + a.sameAs(new Car("Ferrari", "yellow")));
IO.println(new Car("Fiat") + ", " + new Car("Lancia", "blue"));
IO.println("Cars built: " + Car.getCountBuiltCars());
```

Sample run:

```
3
0
10
3
Ferrari yellow (off) | Ferrari yellow (off) | Ferrari red (on)
true false true
Fiat white (off), Lancia blue (off)
Cars built: 5
```

2.3 ★★ Encapsulated bank account

Write `BankAccount`:

- a public constant `MAX_WITHDRAWAL = 1000.0`;
- a `number` assigned automatically (1, 2, 3, … from a private static counter), an `owner` and a `balance`, all private; number and owner never change (`final`);
- constructors `BankAccount(String owner)` (balance 0) and `BankAccount(String owner, double initialBalance)`; a negative initial balance throws `new IllegalArgumentException("...")` and does not use up a number;
- `deposit(double amount)` (amount > 0) and `withdraw(double amount)` (amount > 0, $\leq$ `MAX_WITHDRAWAL`, $\leq$ balance); invalid amounts throw `IllegalArgumentException`;
- getters, a static `getCreatedAccounts()`, and `toString()` like `#1 Alice: 120.0`. No `setBalance`.

Run this `main` (you will write proper tests in 4.2):

```
BankAccount a = new BankAccount("Alice", 100);
BankAccount b = new BankAccount("Bob");
a.deposit(50);
a.withdraw(30);
b.deposit(BankAccount.MAX_WITHDRAWAL);
b.withdraw(BankAccount.MAX_WITHDRAWAL);
IO.println(a);
IO.println(b);
IO.println("Accounts: " + BankAccount.getCreatedAccounts());
```

Sample run:

```
#1 Alice: 120.0
#2 Bob: 0.0
Accounts: 2
```

2.4 ★★ A shop in two packages

Build this small project:

- `it.polito.shop.Product` (public): private final `name` and `price`, a package-visible `int stock`, a package-visible constructor, public getters, and a package-visible `removeStock(int qty)`.
- `it.polito.shop.Discounts` (**not** public): `static double apply(double price, int qty)` returns price × qty, with 10% off when qty ≥ 10.
- `it.polito.shop.Shop` (public): products in an array whose capacity is given to the constructor; `addProduct(name, price, stock)` returns false when full or when the name already exists; `find(name)` returns the `Product` or `null`; `sell(name, qty)` lowers the stock and returns the amount paid (using `Discounts`), or -1 when the product is unknown, qty ≤ 0 or the stock is too low; `stockOf(name)` (0 for an unknown product).
- `it.polito.client.ShopApp`: a `main` whose only import is `import it.polito.shop.*;`.

Compile from the project folder with `javac -d out $(find src -name "*.java")` and run `java -cp out it.polito.client.ShopApp`. Then check that `p.removeStock(1)`, `Discounts.apply(1.5, 10)` and `new Product("x", 1, 1)` written in `ShopApp` are all compile errors.

```
Shop shop = new Shop(10);
shop.addProduct("pen", 1.5, 20);
shop.addProduct("notebook", 4.0, 3);
IO.println(shop.sell("pen", 2));
IO.println(shop.sell("pen", 10));
IO.println(shop.sell("notebook", 5));
IO.println(shop.sell("eraser", 1));
IO.println(shop.stockOf("pen") + " " + shop.stockOf("notebook"));
Product p = shop.find("pen");
IO.println(p.getName() + " " + p.getPrice() + " " + p.getStock());
```

Sample run:

```
3.0
13.5
-1.0
-1.0
8 3
pen 1.5 8
```

2.5 ★★ Number tools: wrappers, Scanner, casts

Write `NumberTools`:

- `int sumInts(String line)`: a `Scanner` on the string sums every integer and skips the other tokens (`hasNext`, `hasNextInt`, `nextInt`, `next`).
- `String stats(String line)`: `count=3 min=10 max=30 avg=20.0`, or `count=0` when there are no integers. Start min/max from `Integer.MAX_VALUE`/`Integer.MIN_VALUE`; the average must not be truncated.

- `String bases(int n)`: `255 = 0b11111111 = 0xff = 0o377` (wrapper methods `Integer.toBinaryString`, `toHexString`, `toOctalString`).
- `int percent(int part, int total)`: the rounded percentage (1 of 3 → 33, 2 of 3 → 67).
- `boolean sameValue(Integer a, Integer b)`: true when the values are equal – it must work for every value, 1000 included.

```
IO.println("sumInts = " + sumInts("10 abc 30 4.5 -2"));
IO.println("stats   = " + stats("10 20 x 30"));
IO.println("stats   = " + stats("no numbers here"));
IO.println("bases   = " + bases(255));
IO.println("percent = " + percent(1, 3) + " " + percent(2, 3));
Integer x = 1000, y = 1000;
IO.println("sameValue = " + sameValue(x, y) + ", but x == y is " + (x == y));
```

Sample run:

```
sumInts = 38
stats   = count=3 min=10 max=30 avg=20.0
stats   = count=0
bases   = 255 = 0b11111111 = 0xff = 0o377
percent = 33 67
sameValue = true, but x == y is false
```

2.6 ★★ Array tools

Write `ArrayTools` (with `import java.util.Arrays;`):

- `int[][] pascal(int n)`: Pascal's triangle as a jagged array (row i has i+1 elements);
- `int[][] transpose(int[][] m)` for a rectangular matrix;
- `void swapRows(double[][] m, int i, int j)` in constant time, even when the rows have different lengths;
- `double average(double... values)` (NaN when called with no arguments);
- `int max(int first, int... others)`, which cannot be called with zero arguments;
- `int[] concatSorted(int[] a, int[] b)`: both arrays together, sorted (use `Arrays.copyOf` and `Arrays.sort`).

```
IO.println(Arrays.deepToString(pascal(5)));
IO.println(Arrays.deepToString(transpose(new int[][]{{1, 2, 3}, {4, 5, 6}})));
double[][] grid = {{1, 2}, {3, 4, 5}};
swapRows(grid, 0, 1);
IO.println(Arrays.deepToString(grid));
IO.println(average(1, 2, 3, 4) + " " + average(new double[]{2, 4}) + " " + average());
IO.println(max(4) + " " + max(9, 3, 15, 5));
int[] merged = concatSorted(new int[]{7, 2, 5}, new int[]{1, 8, 5});
IO.println(Arrays.toString(merged));
IO.println(Arrays.binarySearch(merged, 7));
int[] copy = Arrays.copyOf(merged, merged.length);
IO.println((copy == merged) + " " + Arrays.equals(copy, merged));
int[] filled = new int[4];
Arrays.fill(filled, -1);
IO.println(Arrays.toString(filled));
```

Sample run:

```
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
[[1, 4], [2, 5], [3, 6]]
[[3.0, 4.0, 5.0], [1.0, 2.0]]
2.5 3.0 NaN
4 15
[1, 2, 5, 5, 7, 8]
4
false true
[-1, -1, -1, -1]
```

2.7 ★★ A box office: static, final, factory methods, singleton

Package `ex2_7`. Four classes and a `Main`; the facade is `BoxOffice`.

R1 – **`Money`**, an immutable amount of euros stored as a whole number of cents (`private final long cents`), so that 0.10 + 0.20 is exactly 0.30:

- private constructor; `public static final Money ZERO`, built once;
- `static Money ofCents(long cents)`: `IllegalArgumentException` for a negative value; returns `ZERO` for 0 and a new object otherwise;
- `static Money of(double euros)`: rounds to the nearest cent, then calls `ofCents`;
- `Money plus(Money other)` and `Money times(int qty)` return new objects, built through `ofCents`;
- `long cents()`; `toString()` → `12.50 EUR`.

R2 – **`Codes`**, a utility class (private constructor, only static methods):

- `String code(char prefix, int n)` → `E0007` (`String.format("%c%04d", ...)`);
- `boolean isValid(String code)`: true for exactly one upper-case letter followed by four digits;
- `int number(String code)`: the number inside a valid code (`Integer.parseInt`); `IllegalArgumentException` for an invalid one.

R3 – **`AuditLog`**, a lazy singleton. A static block prints `AuditLog class loaded`, and the private constructor prints `AuditLog created`. It has `getInstance()`, `void record(String message)`, `int count()` and `String last()` (`null` before the first record).

R4 – **`BoxOffice`**, the facade. Write it with `import static ex2_7.Codes.*;`, so that it calls `code(...)`, `isValid(...)` and `number(...)` without the class name.

- `public static final int MAX_SEATS = 500`.
- `String addEvent(String name, int seats, Money price)` returns the codes `E0001`, `E0002`, ... Every box office numbers its own events from 1, as at the exam, where each test builds a new facade and expects the first code. It throws `IllegalArgumentException` if `seats` is not in 1–`MAX_SEATS`, or with the message `duplicate event <name>` if the name is already used in this office. Capacity: 100 events.
- `Money sell(String code, int qty)` sells `qty` tickets, returns their price, and records `sold <qty> x <code>` in the audit log. Errors, as an `IllegalArgumentException` with this message:
  - `invalid code <code>` for a malformed code;
  - `unknown event <code>` for a code that no event of this office has;
  - `not enough seats` when fewer than `qty` seats are left.
- `int available(String code)`.

- `static int officesCreated()`: how many `BoxOffice` objects have been created.

```
IO.println(Money.of(12.5) + " | " + Money.ofCents(199).times(3) + " | "
        + Money.of(0.1).plus(Money.of(0.2)));
IO.println((Money.of(0) == Money.ZERO) + " " + (Money.of(1) == Money.of(1)));
IO.println(Codes.code('E', 7) + " " + Codes.isValid("E0007") + " " + Codes.isValid("e0007")
        + " " + Codes.number("E0042"));
IO.println("start");
BoxOffice turin = new BoxOffice();
BoxOffice milan = new BoxOffice();
String c1 = turin.addEvent("Concert", 100, Money.of(25));
String c2 = turin.addEvent("Opera", 50, Money.of(40.5));
String m1 = milan.addEvent("Concert", 80, Money.of(30));
IO.println(c1 + " " + c2 + " " + m1 + " offices=" + BoxOffice.officesCreated());
IO.println(turin.sell(c1, 3) + " left=" + turin.available(c1));
IO.println(milan.sell(m1, 2) + " left=" + milan.available(m1));
try {
    turin.sell("E0099", 1);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
try {
    turin.addEvent("Opera", 10, Money.ZERO);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(AuditLog.getInstance().count() + " " + AuditLog.getInstance().last());
```

Sample run:

```
12.50 EUR | 5.97 EUR | 0.30 EUR
true false
E0007 true false 42
start
E0001 E0002 E0001 offices=2
AuditLog class loaded
AuditLog created
75.00 EUR left=97
60.00 EUR left=78
error: unknown event E0099
error: duplicate event Opera
2 sold 2 x E0001
```

2.8 ★★ Food delivery: enum, record and a checked exception

Package `ex2_8`, based on the exam of September 2017. The facade is `Delivery`.

R1 – Types.

- `DeliveryException extends Exception`, with a constructor that takes the message.
- `enum Category { STARTER, MAIN_COURSE, DESSERT, DRINK }` with `String label()`: the name with spaces instead of underscores (`MAIN COURSE`).
- `enum Status { NEW, CONFIRMED, PREPARATION, ON_DELIVERY, DELIVERED }` with `Status next()`: the constant that follows (use `ordinal()` and `values()`). `DELIVERED.next()` throws `IllegalStateException`.
- `record MenuItem(String description, Category category, double price, int prepTime)`, whose `toString()` is `[MAIN COURSE] Chinese noodles : 8.50` (use `Locale.US`).
- `record Line(MenuItem item, int qty)`.

R2 – Menu (capacity 50 items).

- `void addItem(String description, Category category, double price, int prepTime)`: `DeliveryException` if the description already exists.
- `MenuItem findItem(String pattern)`: the only item whose description contains `pattern`, ignoring case. `DeliveryException` with the message `no item matches <pattern>` if none matches, or `<k> items match <pattern>` if several do.

R3 – Orders (capacity 100 orders, 10 lines each).

- `int newOrder()`: the ids 1, 2, ... of this `Delivery`.
- `int addToOrder(int orderId, String pattern, int qty)` adds the item found by `findItem`, then returns the quantity of that item in the order. If the order already has a line for the item, replace that line with a new `Line` holding the sum of the two quantities, because records cannot change. `DeliveryException`: `unknown order <id>` for an unknown order, `order <id> is <STATUS>` if the order is not `NEW`.
- `double total(int orderId)`.
- `String showOrder(int orderId)`: one `description, quantity` entry for each `Line` of the order, in the order the lines were created, separated by `"; "` (see the sample run).

R4 – States.

- `Status status(int orderId)`.
- `int confirm(int orderId)`, `int start(int orderId)`, `int deliver(int orderId)` and `int complete(int orderId)` move the order through NEW → CONFIRMED → PREPARATION → ON_DELIVERY → DELIVERED. Each returns the estimated minutes to delivery:
  - after `confirm`: 5 + the longest preparation time in the order + 15;
  - after `start`: the longest preparation time + 15;
  - after `deliver`: 15;
  - after `complete`: 0.

  Compute the minutes with a `switch` on the new status. If the order does not exist or is not in the right state, throw `DeliveryException`, e.g. `order 2 is NEW, expected CONFIRMED`. Write one private method that does the move, and call it from all four.

```
// main declared with: throws DeliveryException
Delivery d = new Delivery();
d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
d.addItem("Green tea", Category.DRINK, 2.5, 2);
d.addItem("Mochi", Category.DESSERT, 3.0, 5);
IO.println(d.findItem("NOODLE") + " | " + d.findItem("tea"));
try {
    d.findItem("n");
} catch (DeliveryException e) {
    IO.println("error: " + e.getMessage());
}
int o1 = d.newOrder(), o2 = d.newOrder();
d.addToOrder(o1, "noodles", 2);
d.addToOrder(o1, "rolls", 1);
IO.println(d.addToOrder(o1, "noodles", 1) + " " + d.showOrder(o1)
        + " total=" + d.total(o1));
IO.println(d.status(o1) + " " + d.confirm(o1) + " " + d.start(o1) + " " + d.deliver(o1)
        + " " + d.complete(o1) + " " + d.status(o1));
try {
    d.start(o2);
} catch (DeliveryException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(Status.NEW.next() + " " + Category.MAIN_COURSE.label() + " "
        + Status.valueOf("ON_DELIVERY").ordinal());
```

Sample run:

```
[MAIN COURSE] Chinese noodles : 8.50 | [DRINK] Green tea : 2.50
error: 3 items match n
3 Chinese noodles, 3; Spring rolls, 1 total=29.5
NEW 40 35 15 0 DELIVERED
error: order 2 is NEW, expected CONFIRMED
CONFIRMED MAIN COURSE 3
```

2.9 ★★★ A trail race: nested classes

Package `ex2_9`, based on the exam of September 2014. The facade is `Race`; errors are `TrailException` (checked, built like `DeliveryException`).

R1 – Runners (capacity 100).

- Inside `Race`, write `public static class Runner`: immutable, with `bib()`, `name()`, `surname()` and `toString()` → `1 Rossi Mario`.
- `int newRunner(String name, String surname)` returns the bibs 1, 2, ...
- `Runner getRunner(int bib)`: `TrailException` with the message `unknown bib <bib>` for an unknown bib.

R2 – Checkpoints (capacity 20).

- `public class Checkpoint` is an **inner** class of `Race`: every checkpoint belongs to one race. It has `name()` and `position()`.
- `Checkpoint addCheckpoint(String name)` creates the checkpoints in route order, at positions 0, 1, ... `TrailException` for a duplicate name.
- `Checkpoint getCheckpoint(String name)`: `TrailException` if the name is unknown.

R3 – Passages.

- `void record(int bib, int minute)`, a method of `Checkpoint`, registers that the runner passed the checkpoint at that minute of the race. It finds the runner with the race's `getRunner`, so an unknown bib throws. It throws `TrailException` with the message `<bib> already passed <checkpoint>` if the runner already passed there.
- Each checkpoint keeps its passages in a linked list of a `private static` nested class `Passage` (`Runner runner; int minute; Passage next;`), sorted by minute; equal minutes keep the order of arrival.
- `String ranking()`, a method of `Checkpoint`, returns the passages in order, as `3 Bianchi Anna (95), 1 Rossi Mario (102)`, or `""` if there are none. Build the string with a local class `Joiner` declared inside `ranking()`. It has `add(String)` and `result()`, and uses the local variable `separator` (`", "`) of `ranking()`.

R4 – `Runner leader()`, a method of `Race`, returns the runner who reached the furthest checkpoint; among those, the first one to get there. It returns `null` before any passage.

```
// main declared with: throws TrailException
Race race = new Race();
int rossi = race.newRunner("Mario", "Rossi");
int verdi = race.newRunner("Luca", "Verdi");
int bianchi = race.newRunner("Anna", "Bianchi");
Race.Checkpoint start = race.addCheckpoint("Courmayeur");
Race.Checkpoint arnad = race.addCheckpoint("Arnad");
race.addCheckpoint("La Thuile");
IO.println(race.getRunner(bianchi) + " | " + arnad.name() + " " + arnad.position());
start.record(rossi, 0);
start.record(verdi, 0);
start.record(bianchi, 0);
arnad.record(rossi, 102);
arnad.record(bianchi, 95);
IO.println(start.ranking());
IO.println(arnad.ranking());
IO.println("leader: " + race.leader());
race.getCheckpoint("La Thuile").record(verdi, 300);
IO.println("leader: " + race.leader());
try {
    arnad.record(bianchi, 120);
} catch (TrailException e) {
    IO.println("error: " + e.getMessage());
}
try {
    arnad.record(9, 130);
} catch (TrailException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
3 Bianchi Anna | Arnad 1
1 Rossi Mario (0), 2 Verdi Luca (0), 3 Bianchi Anna (0)
3 Bianchi Anna (95), 1 Rossi Mario (102)
leader: 3 Bianchi Anna
leader: 2 Verdi Luca
error: 3 already passed Arnad
error: unknown bib 9
```

Part 3 – Characters and strings (J02a)

3.1 ★ Job applications: text utilities

Package `ex3_1`, based on the exam of June 2015. The facade is `Applications`; errors are `ApplicationException`. Capacity: 50 skills, 50 applicants, 10 capabilities each. When a method throws, it must leave the data unchanged.

R1 – Skills. `void addSkills(String... names)` stores every skill trimmed and in lower case. If one of them already exists, also within the same call, it adds nothing and throws `ApplicationException` with the message `duplicate skill <skill>`.

R2 – Applicants. `void addApplicant(String name, String capabilities)`:

- the name is normalized: extra spaces are removed and every word is capitalized (`"  mario ROSSI "` → `"Mario Rossi"`). An applicant with the same normalized name is a duplicate.
- `capabilities` looks like `"java:9,sql:7"`: capabilities separated by commas, each one a skill and a level separated by `:`. Spaces around the parts are allowed, and an empty string means no capabilities. Use `split`.
- Errors, with these messages:
  - `unknown skill <skill>`;
  - `invalid level <text>` for a level that is not an integer from 1 to 10;
  - `invalid capability <text>` for a part without exactly one `:`;
  - `duplicate applicant <name>`.

R3 – Queries.

- `String getCapabilities(String name)`: the applicant's capabilities sorted alphabetically by skill, in the same format (`"java:9,sql:7"`), or `""` if there are none. The name is normalized before the search. `ApplicationException` (`unknown applicant <name>`) if the applicant does not exist.
- `int level(String name, String skill)`: the applicant's level for that skill, 0 if they do not have it.
- `String findApplicants(String pattern)`: the names that contain `pattern` ignoring case, sorted alphabetically and separated by `", "`.

```java
// main declared with: throws ApplicationException
Applications a = new Applications();
a.addSkills("Java", " SQL ", "uml");
a.addApplicant("  mario   ROSSI ", "sql:7, java : 9");
a.addApplicant("anna bianchi", "uml:10");
a.addApplicant("Marta Rossini", "");
IO.println(a.getCapabilities("Mario Rossi") + " | " + a.getCapabilities("ANNA BIANCHI")
        + " | [" + a.getCapabilities("marta rossini") + "]");
IO.println(a.findApplicants("ross") + " | " + a.findApplicants("AN"));
IO.println(a.level("Mario Rossi", "java") + " " + a.level("Anna Bianchi", "java"));
String[] bad = { "java:11", "cobol:5", "java=5", "java:x" };
for (String caps : bad) {
    try {
        a.addApplicant("Test Person", caps);
    } catch (ApplicationException e) {
        IO.println("error: " + e.getMessage());
    }
}
try {
    a.addSkills("python", "JAVA");
} catch (ApplicationException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
java:9,sql:7 | uml:10 | []
Mario Rossi, Marta Rossini | Anna Bianchi
9 0
error: invalid level 11
error: unknown skill cobol
error: invalid capability java=5
error: invalid level x
error: duplicate skill java
```

3.2 ★★ A restaurant: formatted bills

Package `ex3_2`, based on the exam of July 2016. The facade is `Restaurant(String name, int tables)`, with tables of four seats; errors are `RestaurantException`. Capacity: 50 menus, 50 groups.

R1 – Menus and reservations.

- `void addMenu(String name, double price)`: exception for a duplicate name.
- `int reserve(String representative, int people)` reserves ⌈people / 4⌉ tables for the group and returns how many. If not enough tables are free, it reserves nothing, adds `people` to the refused count and returns 0. Exception if the representative already has a reservation.
- `int getRefused()`, `int getUnusedTables()`.

R2 – Orders. `boolean order(String representative, String... menus)` accepts the order only if there are at least as many menus as people, and returns whether it accepted it. An accepted order replaces the previous one. Exception for an unknown representative or menu.

R3 – Bills and payments.

- `String bill(String representative)` returns the receipt; every line ends with `"\n"`:
  - the header `"%s - table for %s (%d people)"`;
  - one line per distinct menu, in the order the menus first appear in the order: the name left-aligned in 20 columns, the quantity in 2, `" x "`, the unit price `%6.2f` and the line total

`%8.2f`;
  - for groups of five or more, a `Service 10%` line with 10% of the subtotal;
  - a `TOTAL` line.

  `Service 10%` and `TOTAL` are left-aligned in 31 columns, followed by the amount in `%8.2f`. Use `Locale.US`, and round the service charge and the total to the cent with `Math.round(x * 100) / 100.0`. Exception if the representative is unknown or has no accepted order.
- `double pay(String representative)` pays the bill and returns its total; exception if there is nothing to pay.
- `double getIncome()`; `String getUnpaid()`: the representatives with an accepted order not paid yet, alphabetical, separated by `", "`.

```
// main declared with: throws RestaurantException
Restaurant r = new Restaurant("Trattoria Da Gino", 3);
r.addMenu("Pizza margherita", 7.5);
r.addMenu("Tiramisu", 4.0);
r.addMenu("Water", 1.0);
IO.println(r.reserve("Rossi", 5) + " " + r.reserve("Bianchi", 6) + " "
        + r.reserve("Verdi", 2) + " refused=" + r.getRefused()
        + " unused=" + r.getUnusedTables());
IO.println(r.order("Rossi", "Pizza margherita", "Tiramisu") + " "
        + r.order("Rossi", "Pizza margherita", "Tiramisu", "Pizza margherita", "Tiramisu",
                "Pizza margherita"));
r.order("Verdi", "Water", "Water");
IO.print(r.bill("Rossi"));
IO.print(r.bill("Verdi"));
IO.println("unpaid: " + r.getUnpaid());
IO.println(r.pay("Rossi") + " " + r.pay("Verdi")
        + String.format(Locale.US, " income=%.2f", r.getIncome())
        + " unpaid: [" + r.getUnpaid() + "]");
```

Sample run:

```
2 0 1 refused=6 unused=0
false true
Trattoria Da Gino - table for Rossi (5 people)
Pizza margherita     3 x   7.50   22.50
Tiramisu             2 x   4.00    8.00
Service 10%                        3.05
TOTAL                             33.55
Trattoria Da Gino - table for Verdi (2 people)
Water                2 x   1.00    2.00
TOTAL                              2.00
unpaid: Rossi, Verdi
33.55 2.0 income=35.55 unpaid: []
```

3.3 ★★ Supplier bids: StringBuilder, speed and the string pool

Package `ex3_3`, based on the exam of June 2017. The facade is `Bids`; errors are `BidException`. Capacity: 100 bids.

R1 – Bids.

- `void addBid(String supplier, int price)`: one bid per supplier, so a second bid from the same supplier replaces the first. `BidException` (`invalid price <price>`) for a price ≤ 0.
- Keep the array always sorted by increasing price and then by supplier name: insert each bid in its place, shifting the others.

- `boolean hasBid(String supplier)`: the names arrive from many sources (literals, `new String(...)`, concatenations, `String.valueOf(char[])`), and all of them must be found.

R2 – Text.

- `String getBids()` → `"s2:100,s1:120,s3:120"`: the bids in order, built with a single `StringBuilder`, with commas only between bids.
- `String formatPrice(int cents)`: `1234567` → `"12,345.67"`, `5` → `"0.05"`. Start from the digits in a `StringBuilder`, pad them with zeros to at least three, then `insert` the decimal point and the thousands commas, working from the right.

R3 – Speed. `String slowReport(int n)` and `String fastReport(int n)` both return `n` copies of `getBids() + "\n"`; the first builds them with `+=` on a `String`, the second with a `StringBuilder`. Time them with `System.currentTimeMillis()`.

```
// main declared with: throws BidException
Bids b = new Bids();
b.addBid("s1", 120);
b.addBid("s3", 120);
b.addBid("s2", 100);
b.addBid("s4", 90);
b.addBid("s4", 150);                          // replaces 90
IO.println(b.getBids());
String half = "s";
String[] names = {
    "s1", new String("s2"), half + "3", String.valueOf(new char[]{'s', '4'}), "s5"
};
String found = "";
for (String n : names) {
    found += b.hasBid(n) ? "Y" : "n";
}
IO.println(found);
IO.println(b.formatPrice(1234567) + " " + b.formatPrice(5) + " " + b.formatPrice(100000));
try {
    b.addBid("s9", 0);
} catch (BidException e) {
    IO.println("error: " + e.getMessage());
}
long t0 = System.currentTimeMillis();
String slow = b.slowReport(10000);
long t1 = System.currentTimeMillis();
String fast = b.fastReport(10000);
long t2 = System.currentTimeMillis();
IO.println("same: " + slow.equals(fast) + ", length " + fast.length());
IO.println("+= " + (t1 - t0) + " ms, StringBuilder " + (t2 - t1) + " ms");
```

Sample run (your times will differ; the order of magnitude is what matters):

```
s2:100,s1:120,s3:120,s4:150
YYYYn
12,345.67 0.05 1,000.00
error: invalid price 0
same: true, length 280000
+= 1260 ms, StringBuilder 2 ms
```

3.4 ★★★ Importing contacts: encodings

Package `ex3_4`. The facade is `AddressBook`; errors are `ImportException`. Capacity: 100 contacts. Run it with `java -Dstdout.encoding=UTF-8 ...` (see the theory notes).

R1 – Import. `int importData(byte[] data, String charsetName)` decodes the bytes with the named charset. The text has one contact per line (`"\n"`), as `name;email`. Empty lines are skipped. It returns how many contacts it added. It throws `ImportException`:

- `bad line <n>: <line>` for a line without exactly one `;` (lines are counted from 1);
- `name too long: <name>` for a name longer than 20 **code points**.

The lines are all checked first: if any line is bad, nothing is added. The length limit and the message use the name after the repair of R2.

R2 – Repair. Before storing a name, `importData` repairs it with `static String repair(String s)`. If `s` contains `'Ã'`, `repair` takes its ISO-8859-1 bytes and decodes them as UTF-8; it returns that result if it contains no `'�'`, and `s` unchanged otherwise. It also returns `s` unchanged if `s` contains a character that ISO-8859-1 cannot represent, such as `€` (`getBytes` would turn it into `?`).

R3 – Export.

- `byte[] export(String charsetName)`: all contacts as `name;email\n` lines, encoded with that charset. `ImportException` with the message `cannot encode <name> in <charset>` if a name cannot be represented; check it with the charset's encoder.
- `static String hex(byte[] bytes)` → `"C3 A9"`: two upper-case digits per byte.
- `String names()`: the names in import order, separated by `", "`.

```
// main declared with: throws ImportException
AddressBook book = new AddressBook();
byte[] utf8 = "José;jose@mail.es\nZoë 😀;zoe@mail.com\n\nPerché;why@mail.it\n"
        .getBytes(StandardCharsets.UTF_8);
IO.println(book.importData(utf8, "UTF-8") + " " + book.names());
String broken = new String("Müller;mueller@mail.de".getBytes(StandardCharsets.UTF_8),
        StandardCharsets.ISO_8859_1);
IO.println(broken + " -> " + AddressBook.repair(broken.split(";")[0]));
book.importData(broken.getBytes(StandardCharsets.ISO_8859_1), "ISO-8859-1");
IO.println(book.names());
IO.println(AddressBook.hex("é€".getBytes(StandardCharsets.UTF_8)) + " | "
        + book.export("UTF-8").length);
try {
    book.export("ISO-8859-1");
} catch (ImportException e) {
    IO.println("error: " + e.getMessage());
}
try {
    book.importData("a;b;c\n".getBytes(StandardCharsets.UTF_8), "UTF-8");
} catch (ImportException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
3 José, Zoë 😀, Perché
MÃ¼ller;mueller@mail.de -> Müller
José, Zoë 😀, Perché, Müller
C3 A9 E2 82 AC | 86
error: cannot encode Zoë 😀 in ISO-8859-1
error: bad line 1: a;b;c
```

Part 4 – JUnit 5 (J03)

At the exam you get a JUnit test class for each requirement (`TestR1_…`, `TestR2_…`), and your grade comes from how many tests pass. In this part you write those tests yourself.

4.1 ★ Exam-style tests for a parking lot

Package `ex4_1`. Copy the class below into your project. It is correct; your job is to write `TestR1_Parking`, which tests it the way the exam's tests do.

```java
package ex4_1;

public class ParkingException extends Exception {
    public ParkingException(String message) { super(message); }
}
```

```java
package ex4_1;

import java.util.Arrays;

public class Parking {
    public static final double HOURLY_FEE = 1.5;
    private final String[] plates;
    private final int[] entry;
    private int inside = 0;
    private int nextTicket = 1;

    public Parking(int spots) {
        plates = new String[spots];
        entry = new int[spots];
    }

    public int freeSpots() { return plates.length - inside; }

    /** Returns the ticket number, or -1 if the parking is full. */
    public int enter(String plate, int minute) throws ParkingException {
        if (find(plate) >= 0) {
            throw new ParkingException("already inside: " + plate);
        }
        if (inside == plates.length) {
            return -1;
        }
        plates[inside] = plate;
        entry[inside] = minute;
        inside++;
        return nextTicket++;
    }
```

```
    /** Returns the fee: HOURLY_FEE for every started hour. */
    public double exit(String plate, int minute) throws ParkingException {
        int i = find(plate);
        if (i < 0) {
            throw new ParkingException("not inside: " + plate);
        }
        int hours = (minute - entry[i] + 59) / 60;
        inside--;
        plates[i] = plates[inside];
        entry[i] = entry[inside];
        plates[inside] = null;
        return hours * HOURLY_FEE;
    }

    /** The plates inside, sorted, separated by ", ". */
    public String plates() {
        String[] copy = Arrays.copyOf(plates, inside);
        Arrays.sort(copy);
        return String.join(", ", copy);
    }

    private int find(String plate) {
        for (int i = 0; i < inside; i++) {
            if (plates[i].equals(plate)) {
                return i;
            }
        }
        return -1;
    }
}
```

Write `TestR1_Parking`. Its `@BeforeEach` creates a `Parking` with 3 spots, and it has one test for each point:

1. the first two tickets are 1 and 2;
2. a new parking has 3 free spots, and each car that enters takes one;
3. a plate that is already inside cannot enter again (`assertThrows`), and the failed attempt changes nothing;
4. when the parking is full, `enter` returns −1;
5. `exit` returns the fee: 1.50 for 1–60 minutes, 3.00 for 61 minutes, 4.50 for three hours; compare with a tolerance;
6. `exit` of a plate that is not inside throws, and the message is `not inside: <plate>`;
7. `plates()` lists the plates inside in alphabetical order, as `AB123CD, ZZ999ZZ`, and is `""` when the parking is empty;
8. a second `Parking` starts again from ticket 1.

Expected: 8 green tests.

4.2 ★★ Testing the food delivery of 2.8, then a suite

Package `ex4_2`, testing the `Delivery` of 2.8 (`import ex2_8.*;`). Write three test classes, each with an `@BeforeEach` that builds a new `Delivery` with the four items of the 2.8 demo.

`TestR1_Menu`, 4 tests:

- `findItem` ignores case;
- `findItem` throws when nothing matches;

- `findItem` throws when two items match;
- `addItem` throws for a duplicate description and changes nothing: `findItem("rolls").toString()` is still `[STARTER] Spring rolls : 4.00`.

`TestR2_Orders`, 5 tests:

- the first two orders are 1 and 2;
- adding the same item twice sums the quantities and returns the sum;
- `showOrder` format;
- `total` (with a tolerance);
- an unknown order id throws in `addToOrder`, `total` and `showOrder` (three `assertThrows` in one test).

`TestR3_Status`, 4 tests:

- a new order is `NEW` (use `assertSame`);
- the four moves return 40, 35, 15 and 0 for an order of noodles and tea;
- a move from the wrong state throws, and the exception's message is `order 1 is NEW, expected CONFIRMED`;
- after `complete`, `confirm` throws again.

Give `TestR3_Status` a static `@BeforeAll` that prints `R3: start` and a static `@AfterAll` that prints `R3: <n> tests`; the count comes from an `@AfterEach`.

Then write `AllTests`, a suite (`@Suite`, `@SelectClasses`) with the three classes.

Finally, an experiment: make the order counter of your `Delivery` `static`, run `AllTests`, and see which tests fail: every test that expects a fixed id, the ids 1 and 2 or the message about order 1, passes only if it happens to be the first to create orders. Then put the counter back.

Expected: `TestR1_Menu` 4 green, `TestR2_Orders` 5, `TestR3_Status` 4, `AllTests` 13, with `R3: start` and `R3: 4 tests` printed around `TestR3_Status`.

4.3 ★★★ Bug reproduction and TDD

Package `ex4_3`.

1. Bug report: *"`Ranking.format(names, scores)` must return `Ann:9,Bob:7,Zoe:7`: decreasing score, ties in alphabetical order, commas only between players, `""` with no players. Instead it returns `Zoe:7Bob:7,Ann:9,`, and it crashes with no players."* The code is:

```
public class Ranking {
    public static String format(String[] names, int[] scores) {
        String[] n = names.clone();
        int[] s = scores.clone();
        for (int i = 0; i < n.length; i++) {            // selection sort by score
            for (int j = i + 1; j < n.length; j++) {
                if (s[j] < s[i]) {
                    String tn = n[i]; n[i] = n[j]; n[j] = tn;
                    int ts = s[i]; s[i] = s[j]; s[j] = ts;
                }
            }
        }
        String r = n[0] + ":" + s[0];
        for (int i = 1; i < n.length; i++) {
            r += n[i] + ":" + s[i] + ",";
        }
        return r;
    }
}
```

Write `RankingTest` with four tests, all with `assertEquals`:

- `{"Zoe", "Ann", "Bob"}` with `{7, 9, 7}` gives `Ann:9,Bob:7,Zoe:7`;
- `{"Zoe", "Bob"}` with `{7, 7}` gives `Bob:7,Zoe:7`;
- empty arrays give `""`;
- `{"Ann"}` with `{9}` gives `Ann:9`.

On the buggy code: 2 failures, 1 error (the crash) and 1 pass; the JUnit 5 console counts failures and errors together, as "3 tests failed". Then fix `format` until all four are green.

2. Test-driven development: `TimeUtils`, the time arithmetic that exams like the trail race need. Write `TimeUtilsTest` **first**, watch it fail, then implement:

   - `static int toMinutes(String hhmm)`: `"08:15"` → 495. The text must be exactly two digits, `:`, two digits, with hours 0–23 and minutes 0–59. Otherwise, `null` included, it throws `IllegalArgumentException`.
   - `static String toText(int minutes)`: 495 → `"08:15"`; `IllegalArgumentException` outside 0–1439.
   - `static int duration(String from, String to)`: the minutes from `from` to `to`, going past midnight if needed (`"23:30"` → `"00:15"` is 45).

   The tests:

   - one test for valid conversions (`"00:00"`, `"08:15"`, `"23:59"`);
   - one for invalid texts (`"8:15"`, `"24:00"`, `"12:60"`, `"ab:cd"`, `"1215"`, `null`);
   - one for `toText`;
   - one for `toText` out of range (−1, 1440);
   - one for `duration`, normal and past midnight.

   Then:

   - add a stub `parseDuration(String s)` (for texts like `"1h30m"`) that throws `UnsupportedOperationException`, and a test for it marked `@Disabled("parseDuration not implemented yet")`;

- add an `exhaustiveCheck` test: for every minute 0–1439, `toMinutes(toText(m)) == m`. It runs only when the environment variable `SLOW_TESTS` is set (`assumeTrue`).

Expected: 5 green, 1 skipped (disabled) and 1 aborted (assumption); with `SLOW_TESTS=1`, 6 green and 1 skipped.

Part 5 – Inheritance, interfaces, lambdas (J04)

5.1 ★ Bank accounts: inheritance

Package `ex5_1`, based on the BankSystem exam. The facade is `Bank`; errors are `BankException`. Capacity: 100 accounts.

- `Account(int number, String owner, double initial)`:
  - private fields and getters (`getNumber`, `getOwner`, `getBalance`);
  - `deposit(double amount)` and `withdraw(double amount)`. `withdraw` throws `BankException` (`insufficient funds on <number>`) unless the `protected boolean canWithdraw(double amount)` allows it; by default it allows it only while the balance stays ≥ 0;
  - `protected void charge(double amount)` subtracts without checks;
  - `double monthlyFee()` → 1.0;
  - `String describe()` → `3,Paul,522.30` (number, owner, balance with two decimals, `Locale.US`), and `toString()` returns `describe()`.
- `CheckingAccount extends Account`: constructor `(number, owner, initial, overdraft)`; `canWithdraw` allows the balance down to −overdraft; fee 2.0; `describe()` adds `,overdraft 500.00`.
- `SavingsAccount extends Account`: constructor `(number, owner, initial, rate)`; fee 0; `addInterest()` deposits `balance * rate / 100`; `describe()` adds `,rate 2.0%`.
- `PremiumAccount extends CheckingAccount`: fee 0; `describe()` adds `,premium` to the checking account's description.
- `Bank`:
  - `int open(String owner, double initial)`, `openChecking(owner, initial, overdraft)`, `openSavings(owner, initial, rate)`, `openPremium(owner, initial, overdraft)`: account numbers 1, 2, … of this bank;
  - `Account getAccount(int number)`: `BankException` (`unknown account <n>`);
  - `void chargeFees()`: every account is charged its `monthlyFee()`;
  - `int countChecking()`: checking accounts, premium ones included (`instanceof`);
  - `SavingsAccount bestSavingsOf(String owner)`: that owner's savings account with the highest balance, or `null` (pattern-matching `instanceof`);
  - `double totalDeposits()`: the sum of all balances.

```
// main declared with: throws BankException
Bank bank = new Bank();
int a = bank.open("Paul", 100);
int c = bank.openChecking("Anna", 50, 500);
int s = bank.openSavings("Paul", 1000, 2);
int p = bank.openPremium("Luca", 0, 1000);
bank.getAccount(c).withdraw(300);
try {
    bank.getAccount(a).withdraw(300);
} catch (BankException e) {
    IO.println("error: " + e.getMessage());
}
((SavingsAccount) bank.getAccount(s)).addInterest();
bank.chargeFees();
for (int n = 1; n <= 4; n++) {
    IO.println(bank.getAccount(n));
}
IO.println(bank.countChecking() + " " + bank.bestSavingsOf("Paul") + " | "
        + bank.bestSavingsOf("Anna"));
IO.println(String.format(Locale.US, "total %.2f", bank.totalDeposits()));
```

Sample run:

```
error: insufficient funds on 1
1,Paul,99.00
2,Anna,-252.00,overdraft 500.00
3,Paul,1020.00,rate 2.0%
4,Luca,0.00,overdraft 1000.00,premium
2 3,Paul,1020.00,rate 2.0% | null
total 867.00
```

5.2 ★★ Customers and orders: equals, hashCode, toString, clone

Package `ex5_2`. The facade is `Shop`; errors are `ShopException`. Capacity: 100 customers, 100 orders.

- `Customer(String name, String email)`, immutable:
  - two customers are equal when their emails are equal ignoring case; `hashCode()` must be consistent with that;
  - `toString()` → `Rossi Giovanni <greds@torino.it>`.
- `Order implements Cloneable`:
  - fields `int id` (package-private, not final: a copy gets a new id), `Customer customer`, `String[] items` and `int[] qty` (10 lines), `int lines`;
  - `void add(String item, int q)`: adding an item that is already in the order adds to its quantity;
  - `public Order clone()` copies the arrays too; the customer can be shared, because it never changes;
  - `toString()` → `#3 greds@torino.it: 2 x Pizza, 1 x Cola`.
- `Shop`:
  - `void addCustomer(String name, String email)`: `ShopException` (`customer already registered: <email>`) if an equal customer exists; check it with `equals`;
  - `int newOrder(String email)` → 1, 2, ..., for the customer with that email. Search with `equals` against `new Customer("", email)`; `ShopException` if not found;
  - `void addItem(int orderId, String item, int q)`;

  - `int repeat(int orderId)`: a copy of the order, made with `clone`, with the next id. Changing the copy must not change the original;
  - `String show(int orderId)`;
  - `static int countDistinct(Object... items)`: how many different objects according to `equals`.

```
// main declared with: throws ShopException
Shop shop = new Shop();
shop.addCustomer("Rossi Giovanni", "greds@torino.it");
shop.addCustomer("Bianchi Anna", "anna@polito.it");
try {
    shop.addCustomer("G. Rossi", "GREDS@Torino.it");
} catch (ShopException e) {
    IO.println("error: " + e.getMessage());
}
Customer x = new Customer("X", "anna@POLITO.it");
Customer y = new Customer("Anna B.", "anna@polito.it");
IO.println(x.equals(y) + " " + (x.hashCode() == y.hashCode()) + " " + x.equals(null) + " "
        + x.equals("anna@polito.it"));
IO.println(Shop.countDistinct(x, y, new Customer("Z", "z@z.it"),
        "anna@polito.it", "anna@polito.it"));
int o1 = shop.newOrder("greds@torino.it");
shop.addItem(o1, "Pizza", 2);
shop.addItem(o1, "Cola", 1);
int o2 = shop.repeat(o1);
shop.addItem(o2, "Pizza", 1);
shop.addItem(o2, "Tiramisu", 3);
IO.println(shop.show(o1));
IO.println(shop.show(o2));
```

Sample run:

```
error: customer already registered: GREDS@Torino.it
true true false false
3
#1 greds@torino.it: 2 x Pizza, 1 x Cola
#2 greds@torino.it: 3 x Pizza, 1 x Cola, 3 x Tiramisu
```

5.3 ★★ Bank operations: abstract classes and Template Method

Package `ex5_3`, based on the BankSystem exam. The facade is `Ledger`; errors are `LedgerException`.

1. Abstract class `Operation`, with a `protected Operation(int date, double amount)` (the date is the day of the year, 1–365), `getDate()` and `getAmount()`:

   - `protected abstract char sign()`;
   - a **final** `toString()`, the template, → `5,500.50+`: the date, a comma, the amount with two decimals (`Locale.US`) and the sign, with no spaces;
   - a final `double effect()`: the amount, negative when the sign is `-`.

   Concrete `Deposit` (`+`) and `Withdrawal` (`-`).
2. The slides' `Sorter`, fixed:

   - `public abstract class Sorter` with the bubble sort `sort(Object[] v)`;
   - `protected abstract int compare(Object a, Object b)`.

   Then `ByDateDesc` (most recent first) and `ByAmountDesc`. Equal elements keep their order, because the bubble sort swaps only when `compare` is positive.

3. `Ledger(String owner, int date, double initial)`: opening the ledger is a deposit. Capacity: 100 operations.
   - `deposit(int date, double amount)` and `withdraw(int date, double amount)`. `withdraw` throws `LedgerException` (`insufficient funds`) if the amount exceeds the balance. As at the exam, if `date` is earlier than the last operation's, the operation takes the last operation's date.
   - `double balance()`.
   - `String[] movements()`: all the operations by decreasing date, through `ByDateDesc`; operations of the same day stay in the order they were made.
   - `String[] deposits()` and `String[] withdrawals()`: by decreasing amount, through `ByAmountDesc`.

```
// main declared with: throws LedgerException
Ledger l = new Ledger("Paul", 1, 500);
l.deposit(10, 250.5);
l.withdraw(5, 100);                // day 5 is before day 10: done on day 10
l.withdraw(20, 40.25);
try {
    l.withdraw(21, 1000);
} catch (LedgerException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(String.join(" ", l.movements()));
IO.println(String.join(" ", l.deposits()) + " | " + String.join(" ", l.withdrawals()));
IO.println(String.format(Locale.US, "%.2f", l.balance()));
Operation op = new Deposit(3, 7);
IO.println(op + " " + op.effect() + " " + new Withdrawal(4, 2.5).effect());
```

Sample run:

```
error: insufficient funds
20,40.25- 10,250.50+ 10,100.00- 1,500.00+
1,500.00+ 10,250.50+ | 10,100.00- 20,40.25-
610.25
3,7.00+ 7.0 -2.5
```

5.4 ★★ Pricing rules: interfaces, static and default methods

Package `ex5_4`. The facade is `Checkout`; errors are `CheckoutException`. Capacity: 50 products.

1. `interface Pricing` with:
   - the abstract method `double apply(double price)`;
   - `default String describe()` → `"custom"`;
   - `default Pricing then(Pricing next)`: a rule that applies this one, then `next`. It is an anonymous class whose `describe()` is `<this> then <next>`;
   - static factories that return objects of **package-private** classes:
     - `none()`: no change, describe `none`;
     - `percentOff(int percent)`: describe `-10%`, `IllegalArgumentException` (`percent must be 0-100: <p>`) outside 0–100;
     - `fixedOff(double amount)`: describe `-5.00`, never below 0.
2. Two interfaces that clash: `interface Member { default String badge() { return "member"; } }` and `interface Student { default String badge() { return "student";`

`} }`. Write `class StudentMember implements Member, Student`, whose `badge()` returns `member+student` by calling both inherited versions.

3. `Checkout`:
   - `void addProduct(String code, double price)`: `CheckoutException` for a duplicate code;
   - `void setRule(String code, Pricing rule)`; without a rule, a product uses `Pricing.none()`;
   - `double price(String code)`: rounded to the cent;
   - `String describe(String code)` → `BOOK 20.00 -10% then -5.00 = 13.00`;
   - `CheckoutException` for an unknown code.

```
// main declared with: throws CheckoutException
Checkout c = new Checkout();
c.addProduct("BOOK", 20);
c.addProduct("PEN", 1.99);
c.addProduct("BAG", 3);
c.addProduct("CAP", 9.5);
c.setRule("BOOK", Pricing.percentOff(10).then(Pricing.fixedOff(5)));
c.setRule("PEN", Pricing.percentOff(33));
c.setRule("BAG", Pricing.fixedOff(5));
Pricing half = new Pricing() {          // an anonymous class
    @Override
    public double apply(double price) { return price / 2; }
};
for (String code : new String[]{"BOOK", "PEN", "BAG", "CAP"}) {
    IO.println(c.describe(code));
}
IO.println(half.apply(9) + " " + half.describe() + " " + Pricing.none().apply(4.5) + " "
        + Pricing.none().describe());
IO.println(new StudentMember().badge());
try {
    Pricing.percentOff(120);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
BOOK 20.00 -10% then -5.00 = 13.00
PEN 1.99 -33% = 1.33
BAG 3.00 -5.00 = 0.00
CAP 9.50 none = 9.50
4.5 custom 4.5 none
member+student
error: percent must be 0-100: 120
```

5.5 ★★★ Menu combos: Composite

Package `ex5_5`. The facade is `Menu`; errors are `MenuException`. Capacity: 50 entries.

- `interface MenuEntry` with `String name()`, `double price()` and `String describe()`, plus a static helper `wrap(MenuEntry e)`: `e.describe()`, in parentheses only if `e` is `Composed`.
- `interface Composed { }`: an empty flag interface.
- A leaf, `record Dish(String name, double price) implements MenuEntry`, whose `describe()` is the name.
- A composite, `class Combo implements MenuEntry, Composed`:

  - constructor `Combo(String name, int discountPercent, MenuEntry... parts)`; `IllegalArgumentException` with no parts or a discount outside 0–50;
  - `price()`: the sum of the parts' prices minus the discount, rounded to the cent with `Math.round(sum * (100 - discount)) / 100.0`;
  - `describe()` → `Lunch: Burger + Fries + (Drinks: Cola + Water)`; the parts are wrapped with `MenuEntry.wrap`;
  - `parts()` returns a copy of the parts.

- `Menu`:
  - `void add(MenuEntry e)`: `MenuException` (`duplicate entry <name>`);
  - `String card()`: one line per entry, in insertion order, `"%-8s%6.2f\n"` (`Locale.US`);
  - `static int countDishes(MenuEntry e)`: how many dishes it contains, counted recursively.

```
// main declared with: throws MenuException
MenuEntry burger = new Dish("Burger", 8), fries = new Dish("Fries", 3);
MenuEntry drinks = new Combo("Drinks", 10, new Dish("Cola", 2.5), new Dish("Water", 1.5));
MenuEntry lunch = new Combo("Lunch", 20, burger, fries, drinks);
Menu menu = new Menu();
menu.add(burger);
menu.add(fries);
menu.add(drinks);
menu.add(lunch);
IO.println(drinks.describe() + " = " + drinks.price());
IO.println(lunch.describe() + " = " + lunch.price());
IO.println(Menu.countDishes(lunch) + " " + Menu.countDishes(fries) + " "
        + (lunch instanceof Composed) + " " + (burger instanceof Composed));
IO.print(menu.card());
try {
    menu.add(new Dish("Lunch", 1));
} catch (MenuException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
Drinks: Cola + Water = 3.6
Lunch: Burger + Fries + (Drinks: Cola + Water) = 11.68
4 1 true false
Burger    8.00
Fries     3.00
Drinks    3.60
Lunch    11.68
error: duplicate entry Lunch
```

5.6 ★★★ Restaurant chain reports: Comparable, Comparator, Strategy

Package `ex5_6`, based on R4 of the exam of July 2016. The facade is `Chain`; errors are `ChainException`. Capacity: 20 restaurants.

- `Restaurant(String name, int tables) implements Comparable<Restaurant>`, natural order by name:
  - `void book(int people)`: takes $\lceil \text{people} / 4 \rceil$ free tables, or, if there are not enough, adds `people` to the refused count;
  - `void pay(double amount)`;

  - `getName()`, `getIncome()`, `getRefused()`, `getUnusedTables()`.

- `Chain`:
  - `void addRestaurant(String name, int tables)`: exception for a duplicate;
  - `Restaurant getRestaurant(String name)`: `ChainException` (`unknown restaurant <name>`);
  - four reports that return the names in order:
    - `String[] byName()`: natural order;
    - `String[] sortByIncome()`: decreasing income, ties by name, with a named class `ByIncomeDesc implements Comparator<Restaurant>`;
    - `String[] sortByRefused()`: increasing refused, ties by name, with an anonymous class;
    - `String[] sortByUnusedTables()`: increasing unused tables, ties by name, with a lambda;
  - `String[] top(int n, Comparator<Restaurant> order)`: the first `n` names in an order chosen by the caller (the Strategy pattern). Restaurants that compare as equal keep the order in which they were added.

No comparator may subtract numbers, as the slides' `StudentCmp` does.

```
// main declared with: throws ChainException
Chain ch = new Chain();
ch.addRestaurant("Gino", 3);
ch.addRestaurant("Arturo", 2);
ch.addRestaurant("Bice", 5);
ch.addRestaurant("Dora", 2);
ch.getRestaurant("Gino").book(10);
ch.getRestaurant("Gino").book(2);
ch.getRestaurant("Arturo").book(9);
ch.getRestaurant("Bice").book(4);
ch.getRestaurant("Dora").book(8);
ch.getRestaurant("Gino").pay(120.5);
ch.getRestaurant("Bice").pay(80);
ch.getRestaurant("Dora").pay(120.5);
IO.println(Arrays.toString(ch.byName()));
IO.println(Arrays.toString(ch.sortByIncome()));
IO.println(Arrays.toString(ch.sortByRefused()));
IO.println(Arrays.toString(ch.sortByUnusedTables()));
Comparator<Restaurant> byNameDesc = Comparator.comparing(Restaurant::getName).reversed();
IO.println(Arrays.toString(ch.top(2, byNameDesc)));
IO.println(Arrays.toString(ch.top(3, Comparator.comparingDouble(Restaurant::getIncome))));
try {
    ch.getRestaurant("Zio");
} catch (ChainException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
[Arturo, Bice, Dora, Gino]
[Dora, Gino, Bice, Arturo]
[Bice, Dora, Gino, Arturo]
[Dora, Gino, Arturo, Bice]
[Gino, Dora]
[Arturo, Bice, Gino]
error: unknown restaurant Zio
```

5.7 ★★★ A train timetable: Iterable and Iterator

Package `ex5_7`. The facade is `Timetable`.

- `record Train(String code, int departure, String destination)`: the departure is in minutes after midnight. `toString()` → `08:15 FR9611 Milano`.
- `Timetable implements Iterable<Train>`:
  - it keeps the trains in a linked list of a private static nested class, sorted by departure; equal departures keep the order in which they were added;
  - `void add(Train t)`, `int size()`;
  - `iterator()` returns an object of a private inner class, whose `next()` throws `NoSuchElementException` at the end;
  - `Iterable<Train> between(int from, int to)`: the trains departing from `from` (included) to `to` (excluded). It is a *view*, not a copy: an anonymous `Iterable` whose iterator, another anonymous class, skips the other trains;
  - `Iterable<Train> to(String destination)`: another view, with the destination compared ignoring case;
  - `Train next(int minute)`: the first train departing at `minute` or later, or `null`. Use a for-each on `this`.

```
Timetable t = new Timetable();
t.add(new Train("RV2021", 9 * 60 + 5, "Aosta"));
t.add(new Train("FR9611", 8 * 60 + 15, "Milano"));
t.add(new Train("R10023", 9 * 60 + 5, "Chivasso"));
t.add(new Train("FR9635", 17 * 60 + 15, "Milano"));
for (Train tr : t) {
    IO.println(tr);
}
String s = "";
for (Train tr : t.between(9 * 60, 12 * 60)) {
    s += tr.code() + " ";
}
IO.println(s.trim());
s = "";
for (Train tr : t.to("MILANO")) {
    s += tr.code() + " ";
}
IO.println(s.trim());
IO.println(t.next(10 * 60) + " | " + t.next(18 * 60));
Iterator<Train> it = t.iterator();
for (int i = 0; i < t.size(); i++) {
    it.next();
}
try {
    it.next();
} catch (NoSuchElementException e) {
    IO.println("no more trains");
}
```

Sample run:

```
08:15 FR9611 Milano
09:05 RV2021 Aosta
09:05 R10023 Chivasso
17:15 FR9635 Milano
RV2021 R10023
FR9611 FR9635
17:15 FR9635 Milano | null
no more trains
```

5.8 ★★★ Order statistics: functional interfaces, lambdas, method references

Package `ex5_8`.

1. `record Order(int id, String customer, double total, boolean delivered)`. In a class `Stats`, write static methods that take the behaviour as a parameter (`java.util.function`):
   - `int count(Order[] v, Predicate<Order> cond);`
   - `double sum(Order[] v, ToDoubleFunction<Order> value);`
   - `Order max(Order[] v, Comparator<Order> order)`, `null` for an empty array;
   - `void forEachIf(Order[] v, Predicate<Order> cond, Consumer<Order> action);`
   - `String[] map(Order[] v, Function<Order, String> f);`
   - `Order[] generate(int n, IntFunction<Order> maker)`.
2. Run the code below. Then replace each lambda on a line marked `// ->` with the equivalent method reference, and the comparator lambda with `Comparator.comparingDouble(...)`. The output must not change.

```
Order[] orders = {
    new Order(1, "Rossi", 29.5, true),
    new Order(2, "Bianchi", 12.0, false),
    new Order(3, "Rossi", 8.0, true),
    new Order(4, "Verdi", 41.25, true)
};
IO.println(Stats.count(orders, o -> o.delivered()) + " "          // ->
        + Stats.count(orders, o -> o.total() > 20));
IO.println(Stats.sum(orders, o -> o.total()));                    // ->
IO.println(Stats.max(orders, (a, b) -> Double.compare(a.total(), b.total())).id());
Stats.forEachIf(orders, o -> o.customer().equals("Rossi"),
        o -> IO.println(o));                                      // ->
IO.println(Arrays.toString(Stats.map(orders, o -> o.customer())));  // ->
IO.println(Arrays.toString(Stats.map(orders, o -> String.valueOf(o.id()))));
Order[] gen = Stats.generate(3, i -> new Order(i + 1, "C" + i, 10 * (i + 1), false));
IO.println(gen[2]);
```

Sample run:

```
3 2
90.75
4
Order[id=1, customer=Rossi, total=29.5, delivered=true]
Order[id=3, customer=Rossi, total=8.0, delivered=true]
[Rossi, Bianchi, Rossi, Verdi]
[1, 2, 3, 4]
Order[id=3, customer=C2, total=30.0, delivered=false]
```

5.9 ★★★ Delivery notifications: Observer by composition (code + test)

Package `ex5_9`.

1. `enum Status { NEW, CONFIRMED, ON_DELIVERY, DELIVERED }`.
2. `Order(int id)`, which starts `NEW` and uses a `java.beans.PropertyChangeSupport` by **composition**:
   - `getId()`, `getStatus()`;
   - `void advance()` moves the order to the next status and fires a `"status"` change (old, new). After `DELIVERED` it throws `IllegalStateException` (`order <id> already delivered`) and fires nothing;
   - `addListener` and `removeListener` delegate to the support object.
3. Listeners:
   - `Dispatcher implements PropertyChangeListener` prints `Courier for order <id>` when the new status is `ON_DELIVERY`. It takes the order from `evt.getSource()`;
   - a **lambda** that prints `Order <id>: <old> -> <new>`.

```
Order o = new Order(7);
Dispatcher d = new Dispatcher();
PropertyChangeListener log = evt -> IO.println("Order " + ((Order) evt.getSource()).getId()
        + ": " + evt.getOldValue() + " -> " + evt.getNewValue());
o.addListener(log);
o.addListener(d);
o.advance();
o.advance();
o.removeListener(log);
o.advance();
try {
    o.advance();
} catch (IllegalStateException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
Order 7: NEW -> CONFIRMED
Order 7: CONFIRMED -> ON_DELIVERY
Courier for order 7
error: order 7 already delivered
```

4. Write `OrderTest` with a recording listener: a lambda that counts the events and stores the last new value in fields of the test class. Three tests:

   - one `advance` sends exactly one event, with the right value;
   - a removed listener receives nothing;
   - after `DELIVERED`, `advance` throws and sends nothing.

   Expected: 3 green tests.